franziskanerinnen
vöcklabruck

Wirtschaftskundliches Realgymnasium und
Oberstufenrealgymnasium der Franziskanerinnen

Schubertstraße 51, 4600 Wels
+43 7242 474 28-30
sekretariat@wrgorg.at / www.wrgorg.at

Neural networks and genetic algorithms using the example of N.E.A.T

Vorwissenschaftliche Arbeit verfasst von

Jakob Zöbl

Klasse: 7B

Betreuungsperson: mag. David Miniberger

Abgabedatum: [Abgabedatum]

Eidesstattliche Erklärung

„Ich erkläre, dass ich die vorwissenschaftliche Arbeit eigenständig angefertigt und nur die im Literaturverzeichnis angeführten Quellen und Hilfsmittel benutzt habe."

_____ _____

Ort, Datum Unterschrift

Abstract

This work focuses on explaining the theoretical aspects of evolutionary computer programs in machine learning as well as neural networks and the combination of both using the example of the algorithm known as “NeuroEvolution of Augmenting Topologies”. The algorithm developed by a team of people lead by Kenneth O. Stanley of the MIT is a revolutionary step in the direction of advanced deep learning. To do this as well as answer the key question literature research has been done. By explaining the different versions and procedures of varying genetic algorithms a basic overview of how artificial intelligence learns is created. To add to this, the general idea of neural networks along with more complex and in detail examples is explained. After understanding both those fundamental concepts of evolutionary computation the next step is merging both of them. This is achieved by taking a closer look at the NEAT-algorithm with a focus on the python implementation. This program is the first of its kind and is the main emphasis of this work

Preface

Because of the ever-growing importance of artificial intelligence in our society as well as my general interest in computer science this topic was just right. Since my personal focus in computation lies in the theoretical aspects and due to my preferred programming language being python, an algorithm that uses ground-breaking new ideas with a focus on its python offshoot fit perfectly. During the process of writing this work and thus by having to learn more about the three main topics my curiosity rose exponentially.

A special thanks goes to my supervisor Mag. David Miniberger who not only supported me by finding and formulating the topic but especially helped me during the writing process and without whom I could not have written this work. Also, I want to thank Mag.ª Eva Edelmann and Mag.ª Sabine Heindl who informed us graduates about all the criteria as well as the writing-style of a pre-scientific-work during countless meetings and workshops. Lastly I want to express my gratitude towards my family who constantly supported me during the course of creating this work.

Wels, 07.03.2026

Jakob Zöbl

Contents

1. INTRODUCTION 1

2. GENETIC ALGORITHMS AND EVOLUTIONARY COMPUTATION IN MACHINE LEARNING 2

2.1 EVOLUTIONARY COMPUTATION 2

2.1.1 Genetic Algorithms 2

2.1.2 Evolutionary Strategies 3

2.1.3 Fitness-Function 3

2.1.4 Termination Criteria 3

2.1.5 Chromosomes 5

2.1.6 Mutation 6

2.1.7 Recombination and Crossover 7

2.1.8 Survival of the fittest 8

2.2 BENEFITS OF GENETIC ALGORITHMS SHOWN BY THE "KNAPSACK-PROBLEM" 9

3. NEURAL NETWORKS 11

3.1 BIOLOGICAL NEURAL NETWORKS 11

3.2 INCORPORATING NEURAL NETWORKS INTO MACHINE LEARNING. 12

3.2.1 Evolution of Mathematical Neural Networks 12

3.2.2 Basics of Neural Networks 12

3.2.3 Explaining the Functions behind Neural Networks 13

3.2.4 Simple Examples of Neural Networks 14

3.3 ADAPTING THE STRUCTURES OF NEURAL NETWORKS 16

3.3.1 Fully connected Neural Networks 16

3.3.2 Not fully connected Neural Networks 17

3.3.3 Avoiding Peeking at the Test Set 18

4. NEUROEVOLUTION OF AUGMENTING TOPOLOGIES 19

4.1 TOPOLOGY AND WEIGHT EVOLVING ALGORITHMS 19

4.2 COMPETING CONVENTION. 19

4.3 SPECIATION 20

4.4 MINIMIZING DIMENSIONALITY 21

4.5 IMPLEMENTATION AND USE-CASES FOR NEUROEVOLUTION OF AUGMENTING TOPOLOGIES 22

4.5.1 NEAT-Python 22

5. CONCLUSION 24

6. BIBLIOGRAPHY 25

7. LIST OF FIGURES 27

8. ANHANG A 29

9. ANHANG B 30

1. Introduction

Artificial intelligence is playing a bigger role in our society than ever before. Most occupational fields today could not exist without some sort of machine learning. This movement towards larger importance of computer science is one that is highly unlikely to stop in the future. On the contrary, the significance is rising exponentially. The largest growing field in computation today is self-learning programs as well as neural networks. For years research has been unsuccessful in their attempt to merge both until Kenneth O. Stanley with a team of MIT employees managed to do so in 2002 by creating NEAT. This results in the title "Neural Networks and Genetic Algorithms using the example of N.E.A.T." While writing my work the main question for me where how programs teach themselves to solve problems on their own, how neural networks aim to imitated natures model and how Kenneth O. Stanley managed to combine both into one innovative algorithm.

The first chapter intends to explain in detail how basic genetic algorithms work. This is done by describing the different components needed to make such a program work as well as how they interact with each other to evolve a program over the course of many generations until a proper solution has been found. Aspects such as the "Fitness-function" or "Termination Criteria" are described. It also shows the benefits of distinct forms of self-improvement-methods such as recombination and mutation. In the second chapter the process of neural networks and their artificial development is made clear. In Addition to that, the various kinds of neural networks are clarified, and two examples are explained. Another big part of this chapter are they possible ways to adapt the structure of artificial neural networks. The last chapter focuses on the NEAT-algorithms and shows how it solves the major problems, such as the "competing-conventions-problem", that previously hindered this blend of genetic algorithms and neural networks. Also, it displays the implementation of NEAT-python in a less complex way and presents general use-cases.

2. Genetic Algorithms and Evolutionary Computation in Machine Learning

2.1 Evolutionary Computation

As Winkler (2008, p. 7) states, the copying of evolutionary behaviours and mechanics follows two main approaches, genetic algorithms (GA) and evolutionary strategies (ES). Whereas genetic algorithms attempt to resemble the biological evolution as a result of the randomness in the mutation (change of values) used in each population, evolutionary strategies are used to obtain a solution as fast and efficient as possible.

The goal is to mutate a given population 0 as many times as necessary until one or more candidates (genotypes) fulfil or succeed a given criteria. ESs spawn every following generation using the values of the best candidates from the previous colony (population) with slight mutations. To calculate how "good" one genotype is compared to all the others, it is most common to use a fitness value. (Winkler 2008, p. 37). This integer is a representation of how close a given individual is from the given criteria. According to Winkler (2008, p. 7) evolutionary strategies also use a different approach in calculating the fitness of each genotype of one generation then GAs.

2.1.1 Genetic Algorithms

Genetic algorithms as well as genetic programming (GP) are used to simulated and imitate a natural evolution hence the mutation is not the main operator unlike in ESs. (Winkler 2008, p. 7). Different from evolutionary strategies, genetic algorithms and programming do not depict each genotype as a vector with mutable variables but rather as a code. This code can be binary, an alphabetical construct, a series of integers, etc. (Adamy 2007, p. 335). They use a fitness value to judge the genotypes in the same way ESs do.

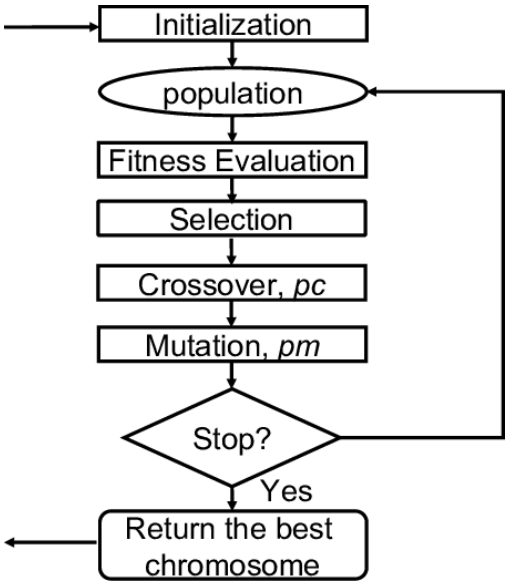

Figure 2-1: The diagram of Standard Genetic Algorithm (Su and Havens, 2014)

All genetic algorithms follow the same pattern. After initializing many possible solutions to one problem, a fitness value is calculated to measure how well each solution works. Then

the best genotypes are used as so-called parents whose chromosomes (values of genotypes) are recombined to create a new individual. The fitness of each solution also resembles its likelihood to be chosen as a parent. Every new solution has a slight mutation though the change is much slighter in comparison to ESs and is exclusively used for avoiding stagnation. The focus is set on recombination instead of mutation. This process continues until a fitting solution is found. (Winkler 2008, p. 8-10), (Adamy 2007, p. 337).

2.1.2 Evolutionary Strategies

Evolutionary strategies are one way of searching for a solution to a problem through evolving existing ones. The emphasis lies in contrast to GAs in the mutations. As with genetic algorithms a generation zero is generated and developed over time. It consists of different solutions which are represented as two vectors. One which contains all the values and one vector of standard deviations. The second one is used to limit the mutations to a certain spectrum so good individuals do not get lost as a result of too drastic changes. Slight recombination can be applied though it is rather uncommon. (Winkler 2008, p.15), (Adamy 2007, p. 302- 303).

2.1.3 Fitness-Function

According to Winkler (2008, p.10) every type of genetic algorithm or program needs to evaluate every member of the population. This is done by calculating the "fitness" of each solution. Typically, the lower the fitness the better. Winkler (2008, p. 10) describes the fitness-value as follows: "…[it] should quantify the adaptedness of a chromosome in correlation with the algorithm's goal. […] the fitness function f is a function $f: S \rightarrow \mathbb{R}^+$ with S being the set of all possible solutions to the problem at hand, also called search space."

2.1.4 Termination Criteria

After a certain amount of k generations evolution will start to stagnate and thus a final best solution has to be chosen. If the assumption is made that a lower fitness value equals a better fitness value, the following formula can be applied to avoid this lack of further improvement. The relative difference of the best and worst candidate solution can be used as an indicator for a minimum in fitness.

$$m = \left| \frac{f(x_{worst}) - f(x_{best})}{f(x_{best})} \right|$$

If the value m then sinks down below a given threshold m_{max} it can be assumed that an optimal solution has been found. $m < m_{max}$

The method is not without its drawbacks. Should the evolution work at a slow but steady rate of improvement the process could be stopped before a true minimum is reached. (Adamy 2007, p. 317).

To avoid running into this problem it is recommended to take the number of generations k and the advancement in a set timeframe q into consideration. If the fitness f does not decrease further in q it is to be assumed that the best genotype is the true minimum. The best individuals from all generations in q are then compared.

- ➔ $f(x_{best}(k))$
- ➔ $f(x_{best}(k-1))$
- ➔ …
- ➔ $f(x_{best}(k-q+1))$

The best solution is called $f_q(x_{best})$ and it is also needed to do the same process to find the worst solution in the timeframe which is $f_q(x_{worst})$. Afterwards the same formula to calculate the relative difference of those two values must be applied.

$$m_q = \left| \frac{f_q(x_{worst}) - f_q(x_{best})}{f_q(x_{best})} \right|$$

The optimisation is then stopped after the threshold $m_{q\,max}$ is reached and $m_q < m_{q\,max}$ is true.

Lastly, another termination criteria can also be set from outside such as a given maximum number of generations or a restriction in processing time.

Annotation: As later mentioned in 2.1.7 “(λ, μ) - evolutionary strategy” allow for the fitness-value to go back up in later generations while “(λ + μ) - evolutionary strategy” are restricted to a constant decline for $f(x)$. (Adamy 2007, p. 318-319)

2.1.5 Chromosomes

As Adamy (2007, p.336) states, genetic algorithms use a codeword to represent one individual. This is in opposition to evolutionary strategies which rely on the usage of vectors as a medium for their solution to be expressed. The codewords do not have to have a specific type of data structure. This representation can be a straight binary code, a real number, a sequence of operators and so on. Adamy (2007, p.336) describes this codeword as the DNS of a genotype. Regardless whether or not the given program is an evolutionary strategy or a genetic algorithm every member of each generation has an assigned fitness-value $f(x)$. According to Winkler (2008, p.10) these chromosomes are also called genomes and are typically n-tuples. All of the values stored are a representation of one piece of information, one characteristic of the candidate solution respectively. Nowadays, the range of options to store and display genotypes is more divers and includes many different data structures.

The most natural way of encoding an individuum in genetic programming is the usage of "hierarchical computer programs of variable sizes" says Winkler (2008, p.24).

Hierarchical computer programs

The most frequent representation is the point-label structure tree. Winkler (2008, p.24) states 4 rules for tree-data-structure with the first one being that every node of the tree has to be a function or a terminal.

Secondly, if the node is a terminal, it is possible to calculate their return value directly without further nodes needed.

The third rule says that the functions are required to have child nodes. Their return value has to be calculated before being used as contributors to the parents' evaluation.

Lastly, rule number 4 states that evaluation of the hierarchical tree is done recursively and through a so called "depth-first way" search. This means that the whole point-label structure tree is being calculated from the root node at the top throughout every following node until the bottom is reached in every branch.

Winkler (2008, p.30) mentions that there are many more ways for candidate solutions to be represented and modelled in a genetic programming process. Regardless, he still states three major reasons in favour of structure trees.

Although they are of rather unusual appearance for many people, programming language compilers can read them with the uttermost ease. In fact, they convert certain regular programs internally into parse tress already. Conventionally however, they are not directly accessible for programmers to view.

Since evaluation is done recursively from top to bottom, meaning starting from the root node going to the lowest one in every branch, it allows for a newly generated candidate solution to be (re-) evaluated almost immediately without a need for any transformations.

Using structure tress also allows for a dynamic and constantly changeable representation of genotypes who keep evolving over time. (Winkler 2008, p. 30-32).

2.1.6 Mutation

By using two vectors it is possible to limit the mutation and fit the range to ones needs. The probability of a mutation is most commonly depicted using a Gaussian probability density function. It shows the likelihood of each value to be mutated. In evolutionary strategies the fitness of one candidate solution does not play a role in selecting the parents but it selects the worst ones which will then be eliminated to provide space for the children. This is one of the major differences to GAs where the fitness is solely determining the parents. (Adamy 2007, p. 303), (Winkler 2008, p.16-17).

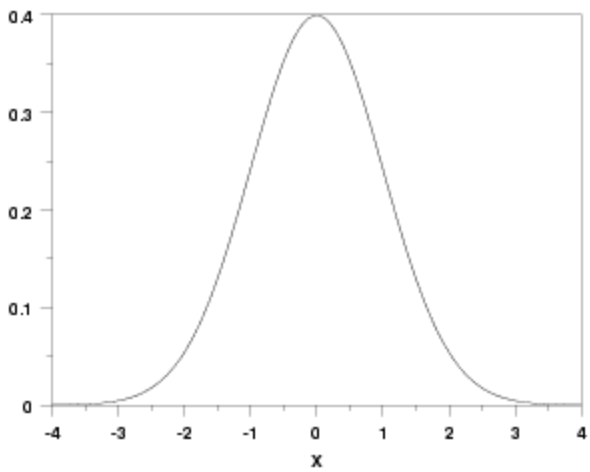

Figure 2-2: [Gaussian probability density function] (no artist, no date)

1.2	0.8	1.1	0.9	1.9	3.4	*Parent*
+0.1	-0.1	+0.2	-0.1	+0.0	+0.4	*Mutation*
1.3	0.7	1.3	0.8	1.9	3.8	*Child*

Figure 2-3: Mutation of a solution of an ES (Zöbl 2021)

2.1.7 Recombination and Crossover

This evolutionary process happens by choosing two genotypes as parents. Their values are then averaged out to generate a child which goes into the next generation of genotypes sometimes along with both parents. μ parents are used to generate λ children (λ ≥ μ). (Winkler 2008, p.16). It is most often seen in genetic algorithms where the fitness is the deciding factor for which individual is picked to become a parent, whereas in ESs its rarer and parents are picked randomly. The approach through recombination instead of mutation as the main source of development makes GAs more similar to nature than evolutionary strategies. (Winkler 2008, p.15). There are two more ways to differentiate between genetic algorithms. Some aim for an extinction of the parent instead of allowing them to reproduce for more than once. If parents do reproduce more than once it is called a "(λ + μ) - evolutionary strategy". With this version it is assured that the fitness-level will always improve from generation to generation. One drawback is that the program is likely to stagnate and reduce the rate of improvement. In contrast to this method is the complete extinction of the parents (also called "(λ, μ) - evolutionary strategy") after a new generation is created. This allows for the fitness to possibly worsen though it is highly unlikely. It also leads to smaller jumps of enhancement but prevent getting stuck in local highs or lows and gives an overall more consistent progress. (Adamy 2007, p. 315-316).

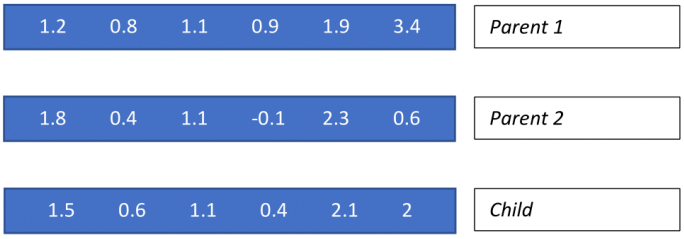

Figure 2-4: Recombination of two solutions in evolutionary computation (Zöbl, 2021)

For more complex problems in which the genotypes cannot be expressed as a single integer it is not possible to calculate an average of two parents since the values of the solutions are saved as so-called chromosomes (2.1.5). For this chromosomes different crossover-methods are used. If the chromosomes are displayed as hierarchical trees, then single-point crossover may get applied. It can be seen as taking a small sub-tree from one parent and swapping it with another sub-tree from the other parent therefore creating two new offsprings. Many different options of choosing the right sub-tree have been come up with over the years. In some cases, it might be more efficient to choose small, large, or even random parts. (Winkler 2008, p. 28-29).

Example: Forming offspring programs using single-point crossover

Figure 2-5 shows how two parents are used to create two completely new solutions. The given example uses single-point crossover to exchange one sub-tree from one parent with one from the other. This leads to vast changes but opens the possibility of producing false solutions. One such example would be if through random crossover a division with zero would occur. This is mathematically impossible thus the neural network could not work, and the given genotype must be terminated. This shows that even in genetic programming certain restrictions need to be considered and implemented in order for the program to work. After completing the change of sub-trees slight mutation is going to be applied. For genetic programming it is implemented through randomly modifying certain nodes of the tree. For hierarchical point-label structure trees this mutation happens in one of two forms. One way is for the program to completely delete or replace a given sub-tree while the other one aims to alter a function nodes function type or change it to a terminal node. The latter one can also go both ways. (Winkler 2008, p. 29-30).

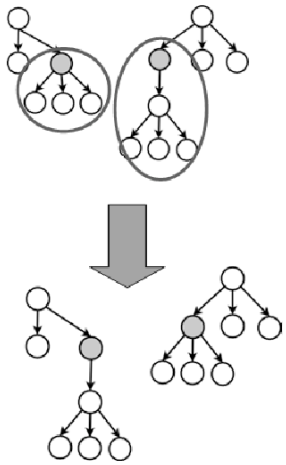

Figure 2-5: One point tree crossover (Avernaso and Taneja, 2006)

2.1.8 Survival of the fittest

As already mentioned in 2.1.6 Mutation and 2.1.7 Recombination and Crossover there are numerous different ways to find fitting genotypes for the role of parents. The main differences between parents in evolutionary strategies and genetic algorithms are the importance of fitness. In brief, in genetic algorithms the likelihood for a given candidate solution to be chosen as a parent is directly proportional to its fitness. On the other hand, evolutionary strategies are more likely to choose random parents since their main method of improvement and evolution is mutation instead of inheritance. (Winkler 2008, p. 15) However, there are multiple ways of comparing genotypes and finding the right one. (Winkler 2008, p. 11)

Proportional Selection

Winkler (2008, p. 11) states that the most widespread form of selection is a proportional one. Each individuals own fitness is compared to the average of its generation to evaluate how much better or worse it is. Afterwards each genotype can be represented as a number

proportional to its fitness. According to Winkler (2008, p. 11) this can be compared to a roulette game where not every field has the same size. Solutions with better fitness have bigger fields and therefore also have a higher chance of the ball falling into their slot. By continuously playing this game, numerous members of one generation will have been picked using somewhat random sampling.

Linear-Rank Selection

The method of linear-rank selection has been described as a system which orders all individuals in regard to their fitness, not proportional but simply numerically. Subsequently the genotypes get copies assigned to them. The better individuals get a predestined multiple of the amount the worse ones get. The beneficial effects of this method include a reduction of domination by so called ”super individuals” which could lead to a stagnation. Also, it enlarges the difference of middle of the pack candidates and thus puts more selective pressure on them in order to prevent reaching local minimums in fitness. (Winkler 2008, p.12)

Tournament Selection

Tournament selection is one umbrella term which could again be split into multiple different tournament models. The most common one, a k-tournament plays out as follows. Whenever one genotype has to be chosen, k candidates are drawn and compared to each other. Only the single fittest one is then picked. This leads to smaller subgroups to improve randomness of evolution and minimize stagnation. (Winkler 2008, p. 12).

Random Selection

This selection model is primarily used for evolutionary strategies and puts no emphasis on fitness whatsoever. Candidate solutions are chosen from one generation completely randomly without taking fitness or any other variable into consideration.

2.2 Benefits of Genetic Algorithms shown by the “Knapsack-Problem”

According to *Genetic Algorithms Explained by Example* (Kie Codes, 2020, 02:48-3:20) the so-called “Knapsack-Problem” presents a backpack alongside different amounts of luggage. Every piece has a given value which shows how important it is and a weight. Only three kilos can be put in the given backpack. Now the best combination of pieces to put into the bag must be found. As stated by *Genetic Algorithms Explained by Example* (Kie Codes, 2020, 08:40-10:23) the time to find the perfect solution increases exponentially and at around 27

possible pieces it is nearly impossible for regular computers to calculate. To shorten the time of calculation a genetic algorithm is used which will reduce the time to closely under two seconds for up to 80 items (with more than 90 percent accuracy) The idea is to display the genotypes (solutions) as integers with the length of pieces to choose from.

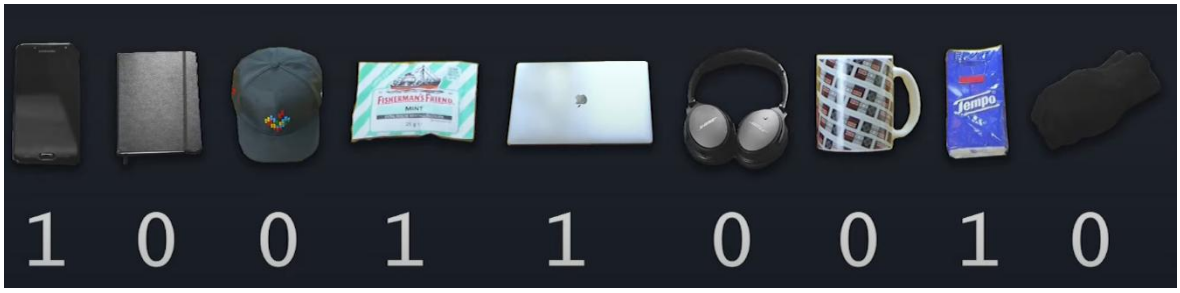

Figure 2-6: [Knapsack-Problem] (Kie Codes, 2020)

As figure 2-6 shows, if the item gets assigned number one it is include and if it gets zero is not. If the weight crosses the limit the fitness is automatically zero and the combination gets terminated. Then the next generation is created through the process of choosing parents, reproducing, and mutating slightly. To keep the process as efficient as possible the program does not aim to find the best solution but rather one of the best. Therefore, it also calculates the possibly of it being the best solution after a certain threshold is reached. (*Genetic Algorithms Explained by Example* (Kie Codes, 2020, 04:17-06:49))

3. Neural Networks

As of today, there is no generally accepted definition of what a neural network is. According to Rey and Wender (2018, p. 15) this is due to there not being one exemplary neural network but rather that it is used as an umbrella term to describe many similar systems. However, what all of them have in common can be defined. They all obtain certain information, process it and put it out afterwards. During this process they are constantly changing and most of the time improving themselves. In the case of artificial neural networks this three steps happen to solve problems. First of all, the network is repeatedly supplied with information in the form of numbers. This are meant as a reality excerpt and start of the process. In the second step this numbers, which are mostly vectors or matrixes, are changed and modified. Lastly, the values are put out in form of useful information again. After evaluating the given output, the network changes itself accordingly. This learning process is repeated for as long as it takes for the output to satisfy a certain termination criterion. (Rey and Wender 2018, p. 15-16). Further explanation of how artificial neural networks work, how they got developed and are implemented follow.

3.1 Biological Neural Networks

Neural networks like in our brain function through a combination of neurons, synapses, and axons. (Adamy 2007, p. 126). They are made up of approximately 10^{11} neurons. The neurons themselves consist of about 10^4 synapses. (Adamy 2007, p. 128). Those neurons are connected to each other with axons and the “bridges” between axons and neurons are called the synapses. They transport stimulus signals, and the synapses decide whether the signal gets amplified or attenuated. The neural network can thus be seen as a sort of filtering algorithm which is calibrated to sort signals based on their importance. (Adamy 2007, p. 126). Their most crucial characteristic remains their ability to adapt to changes. Neural networks learn which signals have a higher priority than others. They change how much synapses amplify or attenuate the transported signals from one neuron to the other, therefore they can improve themselves with experience. (Adamy 2007, p. 128).

3.2 Incorporating Neural Networks into Machine Learning.

3.2.1 Evolution of Mathematical Neural Networks

After breaking it down, on a base level neural networks have n input values with their respective values of x and one or more output variables y. In 1943 the first mathematical model of a neural networks was developed by W. S. McCulloch and W. Pittis. Shortly after, O. Hebb created the first learning procedure in 1949. Six years later, the first working neural networks capable of self-learning and self-improvement got developed and initial computer simulations were run successfully. Since then, a second development wave started around the year 1970 when major parts of the theory of neural networks got improved and expanded. This is seen as the starting point for the increase of use cases after 1985 for advanced computational learning programs such as detecting handwriting. The main reason for their success comes down to their adaptability and ability to find solutions quickly. This learning process, which works through changes in weighting, is illustrated in the figure on the right. If the starting values are picked properly and the learning process is successful it leads to the program being able to work with random values as well and provides useful output data. (Adamy 2007, p. 129-130).

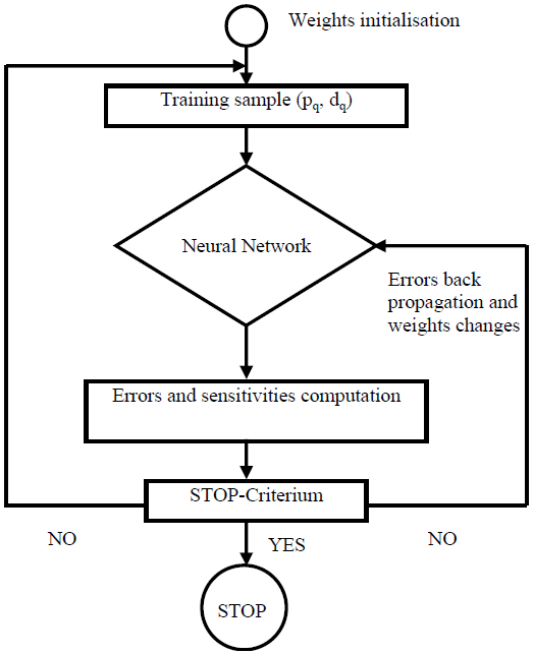

Figure 3-1: Diagram of the learning process of the neural network. (Siddi and Nzeukou, 2014)

3.2.2 Basics of Neural Networks

In an artificial neural network, there is a finite number of layers consisting of "neurons" and each of those neurons is connected to at least one neuron from the next layer through "axons". In the figure on the right, a basic neural network with unlimited layers of neurons is displayed. It also shows a starting layer which is

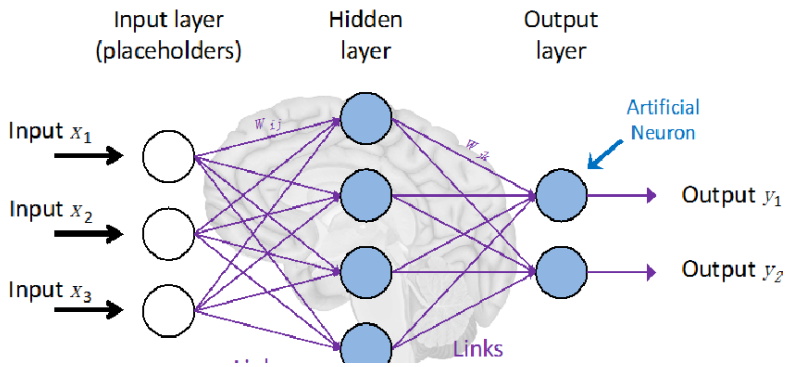

Figure 3-2: Basic neural network design. (Yiming, Liaoyuan, Colin and Harry, 2019)

needed to start of artificial neural networks. This are the so called “input values” in the “input layer”.

3.2.3 Explaining the Functions behind Neural Networks

In *But what is a neural network? | Chapter 1, Deep learning* (3Blue1Brown, 2017, 2:42-8:38) it is explained that artificial neural networks are one large function consisting of many smaller functions. Each individual neuron can be seen as one of those functions. Neurons in the first layer take all inputs from the input layer and use a function to generate a value which is then given over to the next layer of neurons as their starting value. Each value a_0 to a_n from the input layer is assigned a weight w_0 to w_n. The input values are most commonly a number between 0 and 1 while the weights can be any real number leading to this formula: $w_o a_0 + w_1 a_1 + w_2 a_2 + \cdots + w_n a_n$. The outcome of the formula might not be in the range of 0 and 1 thus a sigmoid function ($\sigma\ (x) = \frac{1}{1+e^{-x}}$) is usually applied to keep the values in the wanted range.

According to *But what is a neural network? | Chapter 1, Deep learning* (3Blue1Brown, 2017, 11:34-15:17) every link has a weight assigned to it and every neuron has a bias. This bias can be used to prevent activation or enhance it. For example, if one neuron should only be activated if the weighted sum is greater than 10 the same amount must be subtracted before the sigmoid function is applied. $\sigma(w_o a_0 + w_1 a_1 + w_2 a_2 + \cdots + w_n a_n - 10)$. The artificial neural network evolves by changing the values of weights and biases.

To further simplify the formulas they are preferably depicted as matrices and vectors. The weights can be put into matrices which then get multiplied with the vectors of the values from the previous neurons or the input layer. The biases can be put into a vector as well but instead of multiplying it, it is added or subtracted.

$$\sigma\left(\begin{bmatrix} w_{0.0} & w_{0.1} & \cdots & w_{0.n} \\ w_{1.0} & w_{1.1} & \cdots & w_{1.n} \\ w_{k.0} & w_{k.1} & \cdots & w_{k.n} \end{bmatrix} \begin{bmatrix} a_0^{(0)} \\ a_1^{(0)} \\ a_n^{(0)} \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ b_n \end{bmatrix}\right) \quad \rightarrow \quad a^1 = \sigma(W a^{(0)} + b)$$

(Adamy 2007, p. 137-139).

3.2.4 Simple Examples of Neural Networks

Example: Sorting playing cards

The so called "Wisconsin Card Sorting Test" (WCST) is known as a way of measuring the cognitive capabilities of the prefrontal cortex of an individual. There are many diverse variants of the test. The most common one used is the original one developed by Milner in 1963. It presents the test object with a set of playing cards that need to be put in a certain order. The set consists of cards with that vary in symbols shown, colour of the symbol as well as the number of symbols on the card. Sincere there are four colours, four symbols and a maximum amount of four symbols shown there are exactly 4^3 playing cards per set. Those sixty-four cards are doubled, and another four reference cards are added. The individual is not informed about how to sort the cards. After sorting the first four cards of the deck to the reference cards a supervisor informs them whether it was right or wrong. The intended order is four the one taking the test to first sort cards by colour. After doing this four then rounds meaning the first fourty cards you then have to sort it by shape and lastly by number of symbols. The aim of the artificial neural network developed by Kaplan, Sengör, Gürvit, Genç and Güzelis is to simulate this test and compare the results to humans. This would allow for an accurate measurement of how well a human has done and if their prefrontal cortex is damaged or not. (Rey and Wender 2018, p. 133)

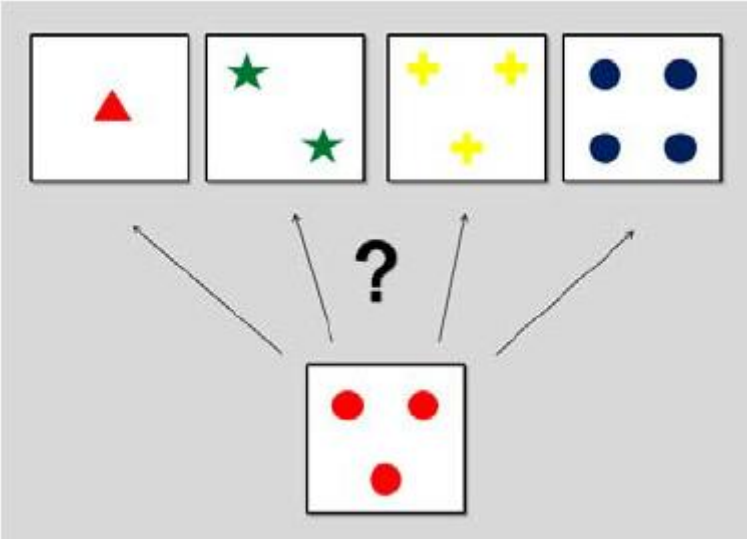

Figure 3-3: Reactive Task-Set Switching Ability, Not Working Memory Capacity, Predicts Cahnge Blindness Sensitivity (Youmans, Robert, Figueroa, Kramarova and Olga, 2011)

The neural network by Alan consists of two separate modules. The first one, the selection module is deciding where the next card should be put. It consists of four neurons where each has three input vectors. One each of the three characteristics of the presented card. All neurons have four nods for their four possible values. For example, the neuron checking the colour has four nods for red, blue, green and yellow.

The second module is again split into two smaller networks. The Hopfield-Network is giving the selection module the feedback. It gets input by a supervisor about the correctness of the assigned cards. It consists of four neurons and is used as a memory between tries.

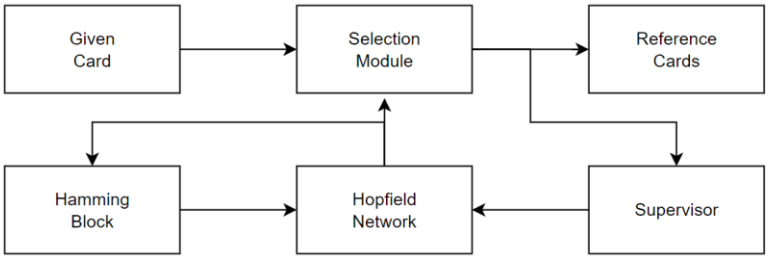

Figure 3-4: Schematic representation of Kaplan's neural network (Rey and Wender, 2018)

The other part of this module is called the Hamming-Block. This network is only active if the selection module failed and produced a wrong outcome. It is responsible for calculating many different ways of sorting the cards. The fourteen neurons of this network each represent one way of sorting. There are more complex ways to sort the stash of cards then just by colour, amount or symbol and all of them can be achieved by combining those fourteen. (Rey and Wender 2018, p. 135)

Example: Detecting Numbers

As stated by *But what is a neural network? | Chapter 1, Deep learning* (3Blue1Brown, 2017, 8:38-13:26) a neural network with the task of recognizing numbers in a black and white picture uses values from 0 to 1 to describe the darkness and or brightness of each pixel. In a small 20 times 20 field there are 400 pixels therefore the neural network would have 400 input values. Each neuron in the following layers would only look at a specific portion of the 400 pixels and with every passing layer it will go into less detail. One possibility would be to have the first layer search for edges, the second for patterns and the output layer for numbers. This method from 3Blue1Brown (2017) links every neuron from one layer to each neuron from the next layer. In order to limit the space one neuron must observe they assign every pixel (input value) outside the subspace a weight of 0 therefore eliminating them from the equation. To detect edges negative weights are assigned to the pixels on the edges

of the field. This leads to higher values and thus more activation in the next neuron if there are any contrasts at all. Also, a bias of -10 is added to only stimulate the next layer if there are hard contrast which will likely be edges.

The second layer is used to take all the edges and put them into constellations. Every neuron has the task of detecting certain forms. For example, one might look for a circle in the upper half while another looks for straight lines in the bottom half.

The last layer, the output layer combines the found shapes and checks if there are any matching digits. This concludes to the neural network having exactly 10 output neurons. The one with the highest output values get active and should show the number entered in the beginning.

3.3 Adapting the Structures of Neural Networks

The way a neural network can evolve is through change of the structure. Besides changes concerning the biases and weights which were already mentioned earlier there are three more changes that can be made in an artificial neural network. In fully connected neural networks, meaning every neuron from one layer is connected to each neuron from the next layer, only the amount of layers and the amount of neurons per layer can be changed. In not fully connected networks the algorithm can change the connection-links between the layers as well. (Russell and Norvig 2010, p. 736-737)

3.3.1 Fully connected Neural Networks

If all the neurons are connected to each other between two connecting layers, the neural network itself will be able to memorise every example almost perfectly. In return this can lead to the network being unable to process new inputs properly. If there are too many parameters in the network (overfitting) it will work better for the already known samples, but it will not be able to generalize well. Figure 3-5 depicts graphs for four similar neural networks with the same task and the same seven input examples but with different number of parameters. In graph b and c overfitting is starting to become a

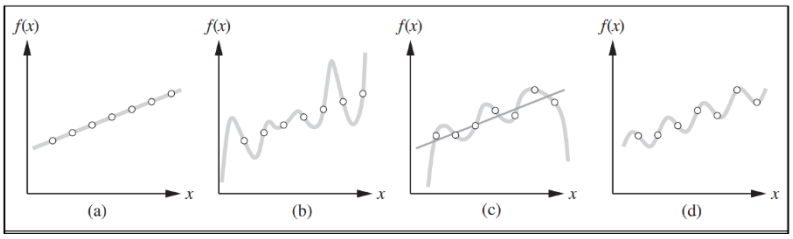

Figure 3-5: Neural Networks with a different number of parameters (Russell and Norvig, 2010)

problem. Even though the graphs of the high-parameter networks align well with the seven inputs, new examples would be far from the wanted outcome. Comparing this to the linear or sinusoidal model from a and b respectively it is apparent that those networks follow a reoccurring pattern and thus they are better suited to process new inputs. (Russell and Norvig 2010, p. 696, 736).

3.3.2 Not fully connected Neural Networks

In not fully connected neural networks neurons from one layer are only connected to some layers from the next and the previous layer. The possible connections are called topologies. To find out what topology delivers the best result a learning process must happen. There are two main approaches to do this. Starting with a fully connected neural network and taking unnecessary links away or to build it up and add to the network. (Russell and Norvig 2010, p. 737).

1. Deleting Unnecessary Connections

One algorithm used for this method is the so called "optimal brain damage" algorithm. It takes a fully connected network and after the initial training round is completed "an information-theoretic approach identifies an optimal selection of connections that can be dropped." (Russell and Norvig 2010, p. 737). After removing the links another test set is run to evaluate whether the systems performance has dropped or not. If not, the process is repeated until the network gets worse. This algorithm also uses the possibility to remove whole neurons if they do not contribute to a lot to find results. This enhances the speed of computation drastically.

2. Adding Needed Links

The reversed approach is to start with a single unit and use it as a foundation to add more neurons to it over time. A common algorithm for this method is the "tiling" algorithm. This method uses the approach of decision list learning. The neural network uses one genome to try and solve all the learning example in the initial training round. Afterwards the least amount of subsequent units needed to work out the unsolved examples are inserted. If every initial problem presented during the training can be solved more learning examples are added therefore presenting a need to add more units to the artificial neural network.

3.3.3 Avoiding Peeking at the Test Set

A common problem for artificial neural networks is to perform better in the learning stage (running many test sets to give the network time to improve itself) then in actual use cases. One way to avoid this is the so called "cross-validation" method.

The error rate of one hypothesis h is defined as "the proportion of times that $h(x) \neq y$ for an (x, y) example." (Russell and Norvig 2010, p. 708). If the hypothesis h performs well during the training sets it does not mean that it will also be able to process more general examples properly. This can be compared to a group of students who perform well on a test if they already saw the answer sheet beforehand therefore making their grades invalid. That is where holdout cross-validation begins. To evaluate the accuracy of the neural network and its hypothesis h the available data in split into two. One half is used for training while the other is used to test the accuracy. The problem with this method is that only half of the available data is used for training thus reducing the potential for better performance in return for more accurate evaluation.

To optimize the ratio of data held out and data used to train the k-fold cross-validation method is used. The data is split into k equal parts. Each subset is used to perform one round of training. Before every round $1/k$ of the sample is held out for training. After k rounds a test is performed with all the held-out parts. Most commonly, 5 and 10 are chosen for k. In return, this method extends the computation time. (Russell and Norvig 2010, p. 708).

4. NeuroEvolution of Augmenting Topologies

The approach to reinforced machine learning by Kenneth O. Stanley are evolving artificial neural networks using the similar methos to genetic algorithms. The program called "NeuroEvolution of Augmenting Topologies", short N.E.A.T, is a topology and weight evolving artificial network. Regular TWEANNs face many problems such as competing conventions, which Kenneth O. Stanley set out to solve with his own algorithmic solution to neural networks. (Stanley and Miikulainen 2002, p. 99).

4.1 Topology and Weight Evolving Algorithms

As explained in "3. Neural Networks" neural networks face the problem of representing its solutions in an evolvable way. The two most ways of encoding such TWEANNs are direct or indirect encoding. While directly encoded networks are focusing on changing the individual nodes of the genotype indirectly encodes ones aim to change connections. Therefore, indirectly encoded neural networks must always be not fully connected neural networks. This allows for smaller genotypes and consequently lower processing time. One of the main problems similar approaches to NEAT face is deconstructive behaviour. This means the crossing-over method harms the efficiency of the network. As a consequence of that, the network does not have the time to fine tune its weights which makes it unpractical to use. Besides that, also long processing time due to large neural networks are limiting real life applications for most topology and weight evolving artificial neural networks. (Stanley and Miikulainen 2002, p. 101, 103).

4.2 Competing Convention.

Regular TWEANNs simply use a combination of crossing-over evolution and random mutation of the weights. The result is a constant lack of improvement due to the randomness in the recombination. The likelihood for a single-point-crossover to lead to an impossible to calculate network is high while the chance of success is low. Figure 4-1 depicts such a crossover scenario. In this case each neuron of the hidden layer (A, B, and C) contain critical information and it would

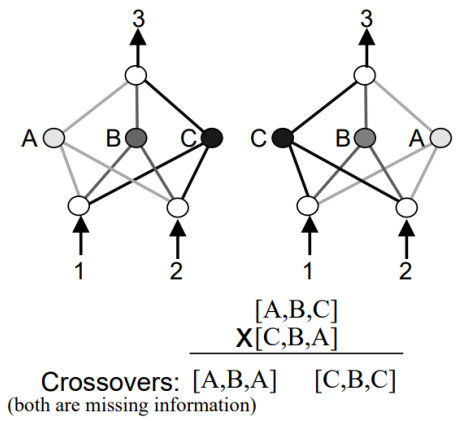

Figure 4-1: Evolving NN's through Augmenting Topologies (Stanley and Miikulainen, 2002)

worsen the network to lose any one of them. This concludes to the offspring [A,B,A] and [C,B,C] to missing important factors. Adding to this is the mutation of weights of the connections between the nods. While for regular artificial neural networks this will eventually lead to improvement as an effect of the fitness function here this is not the case. The timeframe for the weights to properly adjust is too small and thus makes it highly unpractical. One aim of NEAT is to solve the fundamental problem of such vastly different ways of representing neural network structures. Genomes can vary in size or nodes in the same place might state different traits. Other times, nodes that express the exact same trait as others may be located differently. NEAT is trying to imitate natures approach to the same problem. Without keeping crossover evolution in order, the majority of offspring in nature would be decimated. Nature itself utilizes homology. "Two genes are homologous if they are alleles of the same trait." (Stanley and Miikulainen 2002 p. 104). The assessment of this homology between neural networks is the part NEAT solved. Neuro evolution of augmenting topologies uses historical markings in the artificial neural networks as evidence for homology. Whenever two genes share the same origin, they are suitable for artificial synapsis. (Stanley and Miikulainen 2002 p. 103-104).

4.3 Speciation

In nature those different types of structures are assigned to different species which then compete against each other rather then have internal conflict. This leads to natural protection of evolution within a given niche. This niche-system is adapted over to neural network evolution in NEAT. Certain innovative structures are isolated into species which gives them time to optimize on their own. Later they will then compete with the full population to figure the best species out which will then be the starting point to spawn many more new species. In genetic algorithms this is better known a niching. There it is most often used for multimodal function optimization. However, because of the need for a compatibility function to sort the genomes into their respective species and the difficulty such a function along with the competing conventions present it has not been used in a TWEANN. Because NEAT solved the competing conventions problem with the use of historical marking of the neural network it is easy to speciate an entire population. For this explicit fitness sharing is used. This method was first introduced by Holland (1975) and was originally intended to group individuals with ones who have a similar fitness rather then sort them after their genetic similarity. "The result of sharing fitness is that the number of networks that can

exist in the population on a single fitness peak is limited by the size of the peak." (Stanley and Miikulainen 2002 p. 105). Hence the amount of species is restricted by the amount of peaks. This prevents seemingly better species from overtaking the entire population. Consequently, new and innovative solutions in NEAT are isolated with comparable genomes and have time to develop and advance in a protected environment. (Stanley and Miikulainen 2002 p. 104-105). To accurately measure the compatibility of two genomes one must take the excess as well as the disjoint genes into consideration. More disjoint genes have less compatibility since they share less history together which can be quantified by looking at the historical markings. The compatibility distance δ is calculated by using this formula:

$$\delta = \frac{c_1 E}{N} + \frac{c_2 D}{N} + c_3 * \overline{W}$$

Using c_1, c_2 and c_3 allows for proper adjustment of the factors the same way weights work inside the neural network. E and D are the number of excess and disjoint genes respectively while $\overline{W}$ is the average weight discrepancy of two genes. N represents the size of the larger genome or simply put the amount of genes it has. This leads to a normalisation of genome size. The value δ_t is then utilized as a threshold value for genes to sort them into species. Every one species must also have a representative genome, which in the case of NEAT is the first genome assigned to one species. Whenever a new neural network solution g is created through evolution its compatibility is compared to every representative genomes δ value. If none of them fit under the compatibility threshold a new species is generated with g set as the representative genome. (Stanley and Miikulainen 2002 p. 110).

4.4 Minimizing Dimensionality

Most regular TWEANNs use a completely random collection of artificially generated neural networks as their initial population. This is due to it guaranteeing high diversity from the beginning. Nevertheless, it presents major problem for TWEANN systems such as the possibility of creating infeasible networks which need time to die out. Also, many genomes spawn with unnecessary nodes and connection which could be neglected. Since they theoretically do not harm the fitness value there is no need to use procedures such as the "optimal brain damage" algorithm to delete them and shrink the genome. When the genome is larger it takes more time to evaluate and thus needs more processing power. One

major goal of NEAT is a reduction of processing time so larger genomes should in theory be seen as worse. In the case of NEAT this is called “minimizing dimensionality through incrementally growth from minimal structure” (Stanley and Miikulainen 2002 p. 111). Incorporating the size of the genome into the calculation of the fitness value by punishing larger networks and rewarding smaller ones could be a solution for that. (Stanley and Miikulainen 2002 p. 105). One problem that arises using this penalizing approach is that the needed size of the neural network to work depends on the problem it needs to solve. This means for every new implementation of NEAT this value would need to be adjusted and tested. Another method to solve this problem is to make the neuro-evolution itself aim for minimality in its neural networks. To do so not-fully-connected neural networks which are built up from smaller to larger networks are used. They implement similar ideas to the in 3.3.2 discussed “tiling” algorithm to do so. While in other TWEANNs the initial population consists of randomly generated neural networks, in NEAT the first generation has strictly input and output neurons. All nodes and connections in the hidden layers are added afterwards during the evolution. This ensures for the algorithm to only have to look at the smallest search space possible not only in the initial generation or the final solution but throughout all generations. By minimizing search space, it also minimizes processing time and ensures performance gains. The reason why not all TWEANNs apply this idea is that by starting with only the smallest neural networks possible it also lowers diversity. In return this leads to decimation of topological innovation. This is not a problem for NEAT however, since it uses speciation and thus allows for protection of innovation so starting with the smallest possible neural network is not a problem. (Stanley and Miikulainen 2002 p. 106).

4.5 Implementation and Use-Cases for NeuroEvolution of Augmenting Topologies

NEAT-Python is the most commonly used version of NEAT. It is publicly available and does not require or depend on any other python extensions to work. (NEAT-Python documentation, 2015)

4.5.1 NEAT-Python

The python edition of NEAT is a constantly changing and still improving library that aims to simplify the implementation NEAT into any sort of python program. By installing neat-python through using the console command: “pip install neat-python” it can quickly be

obtained on any computer. Other options to gain access to NEAT are also available but less common. (Installation of NEAT-Python, 2015). The main part of using NEAT is changing values in the so called "Configuration File". This .txt file contains values such as the "fitness_criterion" or the "fitness_threshold". All of them can be changed manually and have varying effects of the effectiveness of the evolving of the artificial neural networks. Some of those values such as the "pop_size", which indicates how many individuals will be every given population, have a larger overall impact on the algorithm then others. All the values and their respective description can be found on "https://neat-python.readthedocs.io/en/latest/config_file.html"

To run and implement NEAT in a given python program three different objects are needed. A "neat.config.Config" object which integrates the previously mentioned parameters of the "Configuration File" into the program as well as a "neat.population.Population" object. The latter one is meant to apply the main evolutionary algorithm. This means three things. First it has to evaluate the fitness, then it tests whether one of the termination criterions got fulfilled or not. Lastly, if non has been satisfied it generates the next population of genomes. The last object needed to use NEAT in python is a run function to start the algorithm.

To actually gain insight into the program and its result a "neat.statistics.StatisticsReporter" object can be called. This will show many different statistics of the program such as the average fitness of every population or the best genome of every generation and every species.

Basic NEAT-Python is primarily used for artificially teaching neural networks to play smaller games such as "FlappyBird". Besides this version of NEAT other more advances options for like "HyperNEAT" for instance are used for more complex tasks. (The NeuroEvolution of Augmenting Topologies (NEAT) Users Page, 2015)

5. Conclusion

The way genetic algorithms work is a fundamental concept for our modern society. Most recent softer depends on such programs. Without it no search engine like Google or Bing would be feasible and our daily life would look vastly different from what it is today. Besides the need for evolutionary computation, the demand for artificial neural networks is equally as high. While they started out is simulations of natures behaviour and were used to study our own cognitive evolution, they now far surpassed this point. Nowadays they are applied for only the most complicated of problems. Their ability to solve tasks of varying difficulty with high efficiency is currently unrivalled in the field of artificial intelligence. The downside to neural Networks is their lack of adaptation without user interference. Once they have a grasp of a problem, they can solve it in no time but they most often require some sort of supervisor for them to work properly. This problem could be fixed by combining them with genetic algorithms which have the ability to start from scratch and build up solution over the course of many generations. The final understanding of artificial learning algorithms is that without the work of Kenneth O. Stanley and his fundamentally different approach to evolutionary neural networks, the field of machine learning would still be in its infancy. He paved the way for even more advanced algorithms that are now competing with the newest version of his "NeuroEvolution of Augmenting Topologies" algorithm.

Before starting this work, I was rather inexperienced in artificial intelligence myself though my interest for it was always high. During the course of creating this work I came across numerous new concepts of computation which in return empowered my curiosity concerning the topic. The frequent need to learn new skills for myself was a constant difficulty while writing as a result of the topic requiring a high understanding of computational logic and mathematics.

In conclusion it can be stated that our lives would not be the same without the continuous improvements made in computation and especially in artificial learning programs. The work of Kenneth O. Stanley played a major role is this on-going process of development. Though today the NEAT-algorithms is mostly used for solving low difficulty games or for exemplary purposes it was the first successful step in merging genetic algorithms with artificial neural networks.

6. Bibliography

Primärquellen

Winkler, S.M. (2008) *Evolutionary System Identification*. Johannes-Keples-Univerisät: Trauner Verlag.

Adamy, J., (2007) *Fuzzy Logik, Neurale Netze und Evolutionäre Algorithmen*. 2nd edn. Acchen: Shaker Verlag.

Russell, S.J. and Norvig, P. (2010) *Artificial Intelligence A Modern Approach*. 3rd edn. Upper Saddle River, N.J.: Pentrice Hall.

Rey, G.D. and Wender, K.F. (2018) *Neuronale Netze*. 3rd edn. Hogrefe.

Stanley, K.O. and Miikulainen, R. (2002) 'Evolving Neural Networks through Augmenting Topologies', *The MIT Press Journals,* Evolutionary computation 10(2), pp. 99-127.doi: 10.1162/106365602320169811

Figure 3-4: Rey, G.D. and Wender, K.F. (2018) Schematic representation of Kaplan's neural network. *Neuronale Netze* 3rd edn. Hogrefe, p. 136, diagram.

Sekundärquellen

Onlinequellen

Homepage des Bundesministeriums für Unterricht, Kunst und Kultur zur Vorwissenschaftlichen Arbeit. http://www.ahs-vwa.at/mod/page/view.php?id=9 [Zugegriffen am 15. 06. 2016]

3Blue1Brown (2017) *But what is a neural network? | Chapter 1, Deep learning.* 5 October. Available at: https://www.youtube.com/watch?v=aircAruvnKk&t=686s (Accessed: 9. August 2021).

Kie Codes (2020) *Genetic Algorithms Explained By Example*. 12 July. Available at: https://www.youtube.com/watch?v=uQj5UNhCPuo (Accessed: 9. August 2021).

NEAT-Python documentation (2015). Available at: https://neat-python.readthedocs.io/en/latest/ (Accessed: 03. January 2022)

Installation of NEAT-Python (2015). Available at: https://neat-python.readthedocs.io/en/latest/installation.html (Accessed: 03. January 2022)

The NeuroEvolution of Augmenting Topologies (NEAT) Users Page (2015). Available at: https://www.cs.ucf.edu/~kstanley/neat.html (Accessed: 03. January 2022)

Figure 2-1: Jianhai, S. and Timothy, H. (2014) The diagram of Standard Genetic Algorithm. Available at: https://www.researchgate.net/figure/The-diagram-of-Standard-Genetic-Algorithm_fig1_286518847 (Accessed: 1. November 2021).

Figure 2-3: no artist. (no date) [Gaussian probability density function]. Available at: https://www.itl.nist.gov/div898/handbook/eda/section3/eda3661.htm (Accessed: 1. November 2021).

Figure 2-5: Aversano, L. and Taneja, K. (2006) One point tree crossover. Available at: https://www.researchgate.net/figure/One-point-tree-crossover_fig3_242598999 (Accessed: 1. November 2021)

Figure 2-6: Kie Codes (2020) [Knapsack-Problem]. Available at: https://www.youtube.com/watch?v=uQj5UNhCPuo (Accessed: 9. August 2020)

Figure 3-1: Siddi, T. and Nzeukou, A. (2014) Diagram of the learning process of the neural network Available at: https://www.researchgate.net/figure/Diagram-of-the-learning-process-of-the-neural-network_fig5_279297392 (Accessed: 1. November 2021)

Figure 3-2: Yiming, L., Liaoyuan, Z. Colin, F. and Harry, M. (2019) Basic neural network design. Available at: https://www.researchgate.net/figure/Basic-neural-network-design_fig1_337191767 (Accessed: 1. November 2021)

Figure 3-3: Youmans, Robert, Figueroa, Kramarova and Olga (2011) Reactive Task-Set Switching Ability, Not Working Memory Capacity, Predicts Cahnge Blindness Sensitivity. Available at: https://www.researchgate.net/figure/The-Wisconsin-Card-Sorting-Task-WCST_fig1_270484240 (Accessed: 2. January 2022)

Figure 3-5: Russell and Norvig, (2010) Neural Networks with a different number of parameters. Available at: https://github.com/yanshengjia/ml-road/blob/master/resources/Artificial%20Intelligence%20-%20A%20Modern%20Approach%20(3rd%20Edition).pdf (Accessed: 2. January 2022)

Figure 4-1: Stanley, K.O. and Miikulainen, R. (2002) Evolving NN's through Augmenting Topologies. Available at: http://nn.cs.utexas.edu/downloads/papers/stanley.ec02.pdf (Accessed: 2. January 2022)

7. List of Figures

FIGURE 2-1: THE DIAGRAM OF STANDARD GENETIC ALGORITHM (SU AND HAVENS, 2014) 2

FIGURE 2-2: [GAUSSIAN PROBABILITY DENSITY FUNCTION] (NO ARTIST, NO DATE) 6

FIGURE 2-3: MUTATION OF A SOLUTION OF AN ES (ZÖBL 2021) 6

FIGURE 2-4: RECOMBINATION OF TWO SOLUTIONS IN EVOLUTIONARY COMPUTATION (ZÖBL, 2021) 7

FIGURE 2-5: ONE POINT TREE CROSSOVER (AVERNASO AND TANEJA, 2006) 8

FIGURE 2-6: [KNAPSACK-PROBLEM] (KIE CODES, 2020) 10

FIGURE 3-1: DIAGRAM OF THE LEARNING PROCESS OF THE NEURAL NETWORK. (SIDDI AND NZEUKOU, 2014) 12

FIGURE 3-2: BASIC NEURAL NETWORK DESIGN. (YIMING, LIAOYUAN, COLIN AND HARRY, 2019) 12

FIGURE 3-3: REACTIVE TASK-SET SWITCHING ABILITY, NOT WORKING MEMORY CAPACITY, PREDICTS CAHNGE BLINDNESS SENSITIVITY (YOUMANS, ROBERT, FIGUEROA, KRAMAROVA AND OLGA, 2011) 14

FIGURE 3-4: SCHEMATIC REPRESENTATION OF KAPLAN'S NEURAL NETWORK (REY AND WENDER, 2018) 15

FIGURE 3-5: NEURAL NETWORKS WITH A DIFFERENT NUMBER OF PARAMETERS (RUSSELL AND NORVIG, 2010) 16

FIGURE 4-1: EVOLVING NN'S THROUGH AUGMENTING TOPOLOGIES (STANLEY AND MIIKULAINEN, 2002) 19

8. Anhang A

9. Anhang B